Process Calculi Formalized in Lean 4:
Rho, Pi, MQ, and Modal Mu-Calculus with Cross-Calculus Bridges

Zar Oruži*

March 2026

Abstract

We present a comprehensive Lean 4 formalization of five process-algebraic systems—the ρ-calculus, π-calculus, MQ-calculus, MeTTa-calculus, and modal μ-calculus—together with three cross-calculus bridges: a π-to-ρ encoding with forward simulation and backward admin reflection, an MQ-to-MORK interoperability bridge, and a MeTTa-to-ρ shared-fragment translation with open-map self-bisimulation witness and forward simulation. The formalization totals over 17,500 lines of Lean 4.28 across 55 files with zero `sorry` and zero custom axioms. Shared infrastructure—typeclasses for parallel composition, restriction, structural congruence, and reflexive-transitive closure—eliminates approximately 40% of cross-calculus duplication. Quantale-valued semantics for μ-calculus enables instantiation to probabilistic, fuzzy, tropical, and PLN-evidence domains.

1 Introduction

Process calculi provide compositional models of concurrent, communicating systems. In this paper we report on the formal verification, in the Lean 4 proof assistant [5], of five such calculi and the bridges that connect them.

1. The **ρ-calculus** [13]: a reflective higher-order calculus where names *are* quoted processes, supporting structural congruence, a COMM reduction rule, an executable engine with soundness proof, the Spice calculus [12] for n-step lookahead, and derived replication and restriction.

2. The **π-calculus** [15, 18]: the standard calculus of mobile processes, with a full encoding into the ρ-calculus following Lybech's name-server approach [9], including forward simulation, encoding morphism structure, weak bisimilarity, and backward admin reflection.

3. The **MQ-calculus** [19]: a quantum process calculus where communication *is* measurement, producing binary outcomes governed by Born-rule probabilities. We formalize backend-parametric denotational semantics with a statevector instantiation and an interoperability bridge to MORK [14].

4. The **MeTTa-calculus** [11]: a symmetric reflective higher-order concurrent calculus with backchaining, formalizing the core of the MeTTa language with COMM and REFL rules, an executable reduction engine, a premise-IR adequacy bridge, and a shared-fragment translation into the ρ-calculus with open-map self-bisimulation witness.

*"Oruži" denotes AI-assisted collaboration using Claude Code (Anthropic), ChatGPT, and Codex (OpenAI).

Component	Location	Files	Lines
ρ-calculus	`ProcessCalculi/RhoCalculus/`	11	4,954
π-calculus	`ProcessCalculi/PiCalculus/`	19	9,134
MQ-calculus	`ProcessCalculi/MQCalculus/`	10	1,433
MeTTa-calculus	`ProcessCalculi/MeTTaCalculus/`	9	1,174
Common infra	`ProcessCalculi/Common/`	4	246
Modal μ-calc	`Logic/ModalMuCalculus.lean`	1	360
Quantale sem.	`Logic/ModalQuantaleSemantics.lean`	1	520
Grand total		**55**	**17,821**

Table 1: Formalization summary: 0 sorries, 0 custom axioms throughout.

5. The **modal μ-calculus** [8, 2]: propositional modal logic with least and greatest fixed points. We provide both standard Boolean semantics and quantale-valued semantics, and prove that the Galois connection in ρ-calculus *is* diamond-box duality.

Three bridges tie these together: the π-to-ρ encoding with simulation theorems, an MQ-MORK binary-fold bridge, and a MeTTa-to-ρ shared-fragment translation. All proofs are checked by the Lean 4.28 kernel with zero `sorry` [20].

Table 1 gives line counts; total formalization exceeds 17,500 lines.

2 The Rho-Calculus

The ρ-calculus [13] is a reflective higher-order process calculus in which names are quoted processes: every name x is of the form $@P$ for some process P. This "reflection" dissolves the traditional name/process distinction and raises expressiveness questions beyond purely behavioral formalisms (Section 6). The formalization comprises 11 files totaling 4,954 lines under `ProcessCalculi/RhoCalculus/`.

2.1 Syntax

Process syntax uses a locally nameless representation. The core inductive type `Pattern` (in `Types.lean`, 221 lines) includes:

Listing 1: Rho-calculus process syntax (from `Types.lean`)

```
inductive Pattern : Type where
  | pNil    : Pattern
  | pPar    : Pattern -> Pattern -> Pattern
  | pInput  : Pattern -> Pattern -> Pattern -> Pattern
  | pOutput : Pattern -> Pattern -> Pattern
  | pDrop   : Pattern -> Pattern
  | pQuote  : Pattern -> Pattern
  | pVar    : Nat -> Pattern
  deriving DecidableEq, Repr
```

The `pQuote`/`pDrop` pair implements reflection: quoting a process produces a name, and dropping a name recovers the process.

2.2 Structural Congruence

Structural congruence (`StructuralCongruence.lean`, 287 lines) is formalized as an inductive relation including alpha-equivalence, parallel commutativity and associativity, flattening of nested parallels, and the property that quoting respects structural congruence:

Theorem 2.1 (Quote Respects SC)**.** *If* $P \equiv_{\mathrm{SC}} Q$ *then* $@P \equiv_{\mathrm{SC}} @Q$.

2.3 COMM Rule and Reduction

The COMM rule (`CommRule.lean`, 298 lines) formalizes the core communication step: when an output $\bar{x}\langle y\rangle$ meets an input $x(z).P$, the name y is substituted for z in P. The full reduction relation (`Reduction.lean`, 430 lines) closes COMM under parallel contexts and structural congruence.

2.4 Executable Engine and Soundness

An executable reduction engine (`Engine.lean`, 427 lines) provides a fuel-bounded decision procedure for single-step reduction. A separate soundness proof (`Soundness.lean`, 739 lines) shows that every engine-computed reduction is derivable in the inductive relation:

Theorem 2.2 (Engine Soundness)**.** *If* `engineStep fuel p = some q` *then* `Reduces p q`.

Remark 2.3 (COMM and Predecessor-Finiteness). The COMM rule erases the channel name: the variable n binding the communication channel appears in the LHS but not in the RHS. Consequently, for any fixed target q', infinitely many distinct predecessors exist (one per choice of n), and the ρ-calculus reduction is not predecessor-finite (`not_global_hPredFinite_langReduces_rhoCalc`). A companion formalization [22] identifies a decidable structural sufficient condition (`LangPredFiniteSafe`) under which predecessor-finiteness holds; the ρ-calculus provably fails it (`rhoCalc_not_langPredFiniteSafe`). This is the precise structural reason that global Hennessy–Milner equivalence requires additional quotienting for the ρ-calculus.

2.5 Spice Calculus: Present Moment and Lookahead

The Spice calculus [12] extends ρ-calculus COMM with n-step lookahead (`SpiceRule.lean`, 419 lines; `PresentMoment.lean`, 923 lines). Agents receive the set of states reachable in $\leq n$ steps before committing to an interaction:

$$\texttt{futureStates}(p, n) = \{q \mid p \rightsquigarrow^{n} q\}$$

When $n = 0$ this recovers standard ρ-calculus COMM. The "present moment" ($n = 1$) gives agents awareness of their immediate interaction possibilities. Key theorems include compositional n-step parallel congruence lifting (`ReducesN.par_head`, `ReducesN.par_any_pos`) and `futureStates_par_head`.

2.6 Derived Replication and Restriction

The file `DerivedRepNu.lean` (669 lines) derives replication $!P$ and restriction $(\nu x)\, P$ as definable constructs within the ρ-calculus, proving they satisfy the expected reduction and congruence laws.

3 The Pi-Calculus and the Pi-to-Rho Encoding

The π-calculus [15, 18] is formalized in 19 files totaling 8,967 lines under `ProcessCalculi/PiCalculus/`. We follow the Lybech 2022 encoding [9] of π-calculus into ρ-calculus via a name-server approach.

3.1 Syntax and Reduction

The π-calculus syntax (`Syntax.lean`, 85 lines) and reduction relation (`Reduction.lean`, 405 lines) follow standard presentations [18].

3.2 Lybech 2022 Encoding

The encoding (`RhoEncoding.lean`, 375 lines) maps π-calculus names to ρ-calculus names via a name server that mediates between the flat name space of π-calculus and the reflective name space of ρ-calculus.

3.3 Forward Simulation

The key correctness result is forward simulation (`ForwardSimulation.lean`, 1,387 lines): if a π-calculus process reduces, its encoding reduces (modulo administrative steps) in ρ-calculus.

Theorem 3.1 (Forward Simulation)**.** *If $P \to_\pi P'$ then $[\![P]\!] \to_\rho^* [\![P']\!]$ (up to administrative reductions).*

The theorem name in Lean is `encoding_forward_single_step_rf`.

3.4 Encoding Morphism Structure

The file `EncodingMorphism.lean` (894 lines) establishes that the encoding preserves the algebraic structure of parallel composition, restriction, and nil:

$$[\![P \mid Q]\!] \equiv [\![P]\!] \mid [\![Q]\!], \qquad [\![\mathbf{0}]\!] \equiv \mathbf{0}, \qquad [\![(\nu a)\, P]\!] \equiv (\nu \hat{a})\, [\![P]\!]$$

3.5 Weak Bisimilarity

Weak bisimilarity (`WeakBisim.lean`, 230 lines; `WeakBisimDerived.lean`, 149 lines) formalizes the standard notion from [18]: two processes are weakly bisimilar when every observable reduction step of one can be matched (up to silent τ-steps) by the other, and vice versa. The definition is Prop-valued and coinductive-style: a witness is an explicit relation R that is symmetric, closed under forward and backward lifting, and preserves barbed observability restricted to a finite name set N (following Lybech's N-restricted barbed bisimilarity [9]). `WeakBisimOpenMapBridge.lean` connects this to the open-maps approach of Joyal, Nielsen, and Winskel [7] by showing that weak N-restricted bisimilarity coincides with path bisimilarity over a `BisimulationKit` that instantiates the generalized open-map framework from `CategoryTheory/GeneralizedOpenMaps.lean`.

3.6 Backward Admin Reflection

The largest single file in the formalization, `BackwardAdminReflection.lean` (2,745 lines), proves that administrative reductions introduced by the encoding can be reflected backward: if $[\![P]\!] \to_\rho^* Q$ via admin steps, then Q is structurally congruent to some $[\![P']\!]$. The file now also exposes a fully general backward-simulation endpoint over encoded step-sources: `weak_backward_full_simulation_encodedSC`, together with encoded-source specializations `weak_backward_full_simulation_encode` and `weak_backward_full_`. These theorems flatten the reflected trace decomposition into three explicit branches: reflexive endpoint, admin-progress branch, or reflected π-step branch with head/tail trace data. This completes the backward direction of the simulation argument: every administrative reduction sequence starting from an encoded process eventually reaches an encoded image again.

4 The MQ-Calculus

The MQ-calculus [19] unifies communication and quantum measurement: a COMM event produces a binary measurement outcome governed by Born-rule probabilities. The formalization (10 files, 1,433 lines) covers syntax, operational reduction, backend-parametric denotational semantics, and a cross-calculus bridge to MORK. This extends the communicating quantum processes framework of Gay and Nagarajan [6] with explicit measurement branching in the reduction semantics.

4.1 Syntax

The process grammar uses de Bruijn-style wire indexing and typed gate specifications:

Listing 2: MQ-calculus process syntax (from `Syntax.lean`)

```
inductive GateOp : Type where
  | H | X | Z
  | custom (name : String)
  deriving DecidableEq, Repr

structure GateSpec where
  op : GateOp
  target : Nat
  deriving DecidableEq, Repr

inductive Process : Type where
  | MQNil  : Process
  | MQPar  : Process -> Process -> Process
  | MQNu   : Process -> Process
  | MQGate : GateSpec -> Process -> Process
  | MQOut  : Nat -> Process
  | MQIn   : Nat -> Process -> Process -> Process
  deriving DecidableEq, Repr
```

The `MQIn` constructor takes two continuations—one for each measurement outcome—making measurement branching explicit in the syntax.

4.2 Communication as Measurement: The COMM Rule

The COMM rule (`CommRule.lean`, 87 lines) produces a binary measurement outcome. A central theorem provides a constructive both-outcomes witness:

Listing 3: COMM rule with constructive both-branches (from `CommRule.lean`)

```
inductive Outcome : Type where
  | zero : Outcome
  | one  : Outcome
  deriving DecidableEq

inductive CommReduction :
    Nat -> Process -> Process -> MeasurementBranch -> Prop where
  | outcome_zero (i : Nat) (p q : Process) :
      CommReduction i p q { outcome := .zero, result := p }
  | outcome_one (i : Nat) (p q : Process) :
      CommReduction i p q { outcome := .one, result := q }
```

```
theorem comm_both_outcomes (i : Nat) (p q : Process) :
    CommReduction i p q { outcome := .zero, result := p }
    /\ CommReduction i p q { outcome := .one, result := q } :=
  (.outcome_zero i p q, .outcome_one i p q)
```

Born-style probabilities are defined separately on quantum state vectors; the branch relation itself remains structural and proof-friendly.

4.3 Backend-Parametric Denotational Semantics

The semantics (`Backend.lean`, 380 lines; `Denotational.lean`, 229 lines) is parameterized by a backend interface that provides gate action, fresh allocation, branch probabilities, and post-measurement collapse:

Listing 4: Backend interface (from `Backend.lean`)

```
structure MQSemanticsBackend where
  branchProb      : Nat -> Outcome -> QState -> Real
  branchProb_nonneg : forall i out st, 0 <= branchProb i out st
  branchProb_sum_one : forall i st,
    branchProb i .zero st + branchProb i .one st = 1
  applyGate       : GateSpec -> QState -> QState
  allocFresh      : QState -> QState
  collapse        : Nat -> Outcome -> QState -> QState
```

The current `statevectorBackend` instantiation gives executable semantics while preserving theorem-level invariants. A key normalization result ensures physical consistency:

Theorem 4.1 (Post-Collapse Normalization)**.** *`collapseByOutcome_norm_eq_one_of_raw_pos`: if* $0 < \textit{rawOutcomeProb}(i, out, st)$ *then* $\|\psi'\| = 1$ *after collapse.*

4.4 MQ-MORK Bridge

MQ measurement branching is linked to MORK binary-fold branching (`Interoperability.lean`, 37 lines) via:

Theorem 4.2 (MQ-MORK Interoperability)**.** *`comm_nondeterminism_iff_mork_binary`: the two COMM outcomes of a measurement correspond bijectively to the two sub-results of a MORK binary fold step.*

This ties communication-level nondeterminism in MQ directly into the MORK three-phase execution model [14].

Executable witness. Two MM2 scripts exercise the bridge concretely on the MORK runtime. MORK operates on a fact database: an `exec` rule matches facts against a pattern (after the comma) and rewrites them via a template (`(+ ...)` adds a fact, `(- ...)` removes one). Both scripts set up identical initial facts and an identical fold rule, differing only in which branch is selected.

Script 1 (`binary_fold_pick0.mm2`)—selects branch 0:

Listing 5: Pick outcome 0 (from `binary_fold_pick0.mm2`)

```
; Initial facts: a pending query with two candidate sub-results
(wait q1)
(subResult q1 0 P)
```

```
(subResult q1 1 Q)

; Choose branch 0 in this run
(pick q1 0)

; Fold rule: when (wait $q), (pick $q $i), and (subResult $q $i $r)
; all match, emit (commOutcome $q $i $r) and consume the wait token
(exec (0 fold)
  (, (wait $q)
     (pick $q $i)
     (subResult $q $i $r))
  (O (+ (commOutcome $q $i $r))
     (- (wait $q))))
```

The pattern binds `$q`=`q1`, `$i`=`0`, `$r`=`P` (since `(pick q1 0)` selects index 0). The template emits `(commOutcome q1 0 P)`.

Script 2 (`binary_fold_pick1.mm2`)—selects branch 1:

Listing 6: Pick outcome 1 (from `binary_fold_pick1.mm2`)

```
; Same initial facts
(wait q1)
(subResult q1 0 P)
(subResult q1 1 Q)

; Choose branch 1 in this run
(pick q1 1)

; Same fold rule
(exec (0 fold)
  (, (wait $q)
     (pick $q $i)
     (subResult $q $i $r))
  (O (+ (commOutcome $q $i $r))
     (- (wait $q))))
```

Now `$i`=`1` and `$r`=`Q`, so the template emits `(commOutcome q1 1 Q)`.

The two outputs—`(commOutcome q1 0 P)` and `(commOutcome q1 1 Q)`—are the operational witnesses for `mork_mq_nondeterminism_corresponds`: the *same* fold rule, applied to the *same* query with the *same* two candidate sub-results, produces *either* outcome depending solely on the branch selection. This is precisely the structural binary nondeterminism that `comm_nondeterminism_iff_mork_binary` formalizes in Lean: MQ's two `CommReduction` outcomes (zero$\to P$, one$\to Q$) biject with MORK's two `FoldPicksSubResult` choices (`subResult0`, `subResult1`).

5 The MeTTa-Calculus

The MeTTa-calculus [11] formalizes the core of the MeTTa language as a symmetric reflective higher-order concurrent calculus with backchaining. Two inputs communicate by unifying their patterns, and patterns bind variables. Read without the variable convention, the communication rule lets one input's bound name capture the other input's free name. The formalization therefore states Comm with a principal, idempotent unifier and with the two inputs opened apart, quotes every pattern variable in the continuations, computes the rule on a locally nameless closed form, and proves each adequacy claim on a stated fragment.

5.1 Syntax

The process grammar is:

$$P, Q ::= \mathbf{0} \mid P \mid Q \mid \mathsf{for}(t \leftarrow x)\, P \mid x\,?\,P \mid *x$$

$$x, y ::= @(P) \qquad t, u ::= (P) \mid \texttt{true} \mid \texttt{false} \mid n \mid s \mid \texttt{sym}$$

where $\mathsf{for}(t \leftarrow x)\, P$ is guarded input (listen for pattern t on channel x, then continue as P), $x\,?\,P$ is reflection (inspect P and emit its future state on x), $@(P)$ quotes a process into a name, and $*x$ drops (unquotes) a name back to a process.

`Syntax.lean` (168 lines) encodes the grammar as abbreviations of the shared `Pattern` AST. Pattern variables are its free variables:

Listing 7: MeTTa-calculus syntax (from `Syntax.lean`)

```
abbrev Proc := Pattern   -- processes
abbrev Name := Pattern   -- names (quoted processes)
abbrev Term := Pattern   -- terms (data values)

def pZero    : Proc := .apply "MZero" []
def pPar     (elems : List Proc) : Proc :=
  .collection .hashBag elems none
def nQuote   (p : Proc) : Name := .apply "MQuote" [p]
def pDrop    (x : Name) : Proc := .apply "MDrop" [x]
def pFor     (t : Term) (x : Name) (k : Proc) : Proc :=
  .apply "MFor" [t, x, k]
def pReflect (x : Name) (p : Proc) : Proc :=
  .apply "MRef" [x, p]
```

Two language definitions are provided: `mettaCalc` (the Comm, Refl and parallel rules) and `mettaCalcCommOnly` (the Comm-only fragment, which the premise contract of Section 5.5 uses as the lookahead of Refl).

Binding and alpha-equivalence. A guard $\mathsf{for}(t \leftarrow x)\, P$ binds every variable of its pattern t, in t and in P; the channel x is outside its scope. A guard quoted inside a pattern binds its own variables. Alpha-equivalence is decided on a locally nameless closed form [4, 3]: `close` replaces every bound occurrence by a code for the depth of its guard and its position in that guard's pattern, and $P \equiv_\alpha Q$ is $\texttt{close}\, P = \texttt{close}\, Q$ (`AlphaEq`). Renaming a bound variable y of a guard to z preserves the closed form (`alpha_alphaEq`) when z occurs neither in the pattern nor in the continuation and y is not rebound inside the continuation. The manuscript requires only $z \notin \mathrm{FN}(P)$; without $z \notin t$ the renaming merges two variables of one pattern and changes the guard's reducts (`alpha_side_condition_needed`).

5.2 Structural Congruence

The manuscript's structural congruence ($\equiv$) is the smallest equivalence closed under:

$$P \mid \mathbf{0} \equiv P \qquad \text{(Nil)}$$

$$P \mid Q \equiv Q \mid P \qquad \text{(Comm)}$$

$$(P \mid Q) \mid R \equiv P \mid (Q \mid R) \qquad \text{(Assoc)}$$

$$@(*x) \equiv x \qquad \text{(QuoteDrop)}$$

`StructuralCongruence.lean` (67 lines) defines the inductive relation `SC` (notation $\approx_m$) with 11 constructors: the equivalence closure, a constructor `alpha` for syntactic identity, the singleton, unit, commutativity and associativity laws of the bag, permutation, and pointwise congruence. QuoteDrop is an equation of the language definition `mettaCalc`. Alpha-equivalence is not a constructor of `SC`; it is `AlphaEq`, and the corrected relational rules below use it as their equivalence.

Listing 8: Structural congruence (from `StructuralCongruence.lean`)

```
inductive SC : Proc -> Proc -> Prop where
  | refl   (p : Proc) : SC p p
  | symm   (p q : Proc) : SC p q -> SC q p
  | trans  (p q r : Proc) : SC p q -> SC q r -> SC p r
  | alpha  (p q : Proc) : p = q -> SC p q
  | par_singleton (p : Proc) : SC (pPar [p]) p
  | par_nil_left  (p : Proc) : SC (pPar [pZero, p]) p
  | par_nil_right (p : Proc) : SC (pPar [p, pZero]) p
  | par_comm  (p q : Proc) :
      SC (pPar [p, q]) (pPar [q, p])
  | par_assoc (p q r : Proc) :
      SC (pPar [pPar [p,q], r]) (pPar [p, pPar [q,r]])
  | par_perm  (xs ys : List Proc) :
      xs.Perm ys -> SC (pPar xs) (pPar ys)
  | par_cong  (xs ys : List Proc) :
      xs.length = ys.length ->
      (forall i hx hy, SC (xs.get {val:=i,isLt:=hx})
                          (ys.get {val:=i,isLt:=hy})) ->
      SC (pPar xs) (pPar ys)
```

Paper-mapped equivalence theorems:

- `paper_equiv_par_nil_left`: $\mathbf{0} \mid P \approx_m P$
- `paper_equiv_par_nil_right`: $P \mid \mathbf{0} \approx_m P$
- `paper_equiv_par_comm`: $P \mid Q \approx_m Q \mid P$
- `paper_equiv_par_assoc`: $(P \mid Q) \mid R \approx_m P \mid (Q \mid R)$

5.3 Reduction Semantics

The capture-avoiding Comm. Two guarded inputs on the same channel communicate when their patterns unify:

$$\frac{\sigma \text{ a principal idempotent unifier of } t \text{ and } u \qquad \text{the two inputs apart}}{\mathsf{for}(t \leftarrow x)\, P \mid \mathsf{for}(u \leftarrow x)\, Q \;\longrightarrow\; P\dot{\sigma} \mid Q\dot{\sigma}} \qquad \text{(Comm)}$$

Here $\dot{\sigma}$ is *dot-substitution*: each pattern variable v becomes the name $@(v\sigma)$, the quotation of its value, so a variable that σ leaves unbound becomes $@v$. This *uniform dot* is the manuscript's dot of a principal binding list that names every pattern variable, a bound one with its value and an unbound one with itself (`completeBindings`, `applyDot_completeBindings`). Because σ is idempotent, the dot is applied once to each continuation. The two inputs are *apart* under the variable convention [1, 21] (`Apart`): the two patterns share no variable, neither input mentions the other's pattern variables, and no guard inside a continuation rebinds its own input's pattern variables. As the convention prescribes, bound names are renamed apart, up to alpha-equivalence, before the rule applies.

The relational rules. `Corrected.Step admit` has the manuscript's four rules: Comm with a principal unifier and an admission on the redex, Par, Equiv by $\equiv_\alpha$, and Refl, whose premise is the same relation, so that reflection is recursive:

$$\frac{P \longrightarrow P'}{x\,?\,P \;\longrightarrow\; \mathsf{for}((P') \leftarrow x)\,\mathbf{0}} \qquad \text{(Refl)}$$

The admission `conventional` is `Apart`. The admission `uniform`, the semantics, adds that the binding list names every pattern variable (`Complete`), so that the dot is uniform; `uniformCalc` restricts it to guard-free patterns and to patterns and continuations of the calculus fragment. The admission `closing` adds to `conventional` that both patterns are guard-free, that patterns and continuations belong to the calculus fragment, and that the unifier binds every pattern variable a continuation reads to a ground term. `Principal t u bs` says that one pass of `bs` makes t and u equal and that every unifier of t and u absorbs it.

Unification. `Corrected.unify` encodes patterns injectively as first-order terms and runs a Martelli–Montanari solver [10] with an occurs check. The solver is evaluated by the kernel and proved equal to a total unification algorithm, and `unify` returns each moved variable with its final value. It is sound, most general and idempotent, and it fails exactly when no substitution unifies the two patterns [17].

The executable step. `cstep` P works on `close` P. At a binary composition of two guards on one channel with guard-free patterns, it opens the two inputs with distinct names fresh for the whole process, unifies the opened patterns, completes the unifier with the opened variables it leaves unbound, and applies the dot once to each opened continuation. It also steps the left component of a composition, and it steps a reflected process with the same function, so it has the nested reflection result.

Listing 9: The corrected COMM (from `CorrectedReduction.lean`)

```
def commClosed (avoid : List String) (first second : Pattern) :
    List Pattern :=
  match asGuard first, asGuard second with
  | some (t, x, k), some (u, y, l) =>
      if x = y /\ guardFree t = true /\ guardFree u = true then
        let leftNames  := freshNames avoid 0 (codeCount t)
        let rightNames := freshNames avoid (codeCount t) (codeCount u)
        match unify (openPattern leftNames t)
                    (openPattern rightNames u) with
        | some bs =>
            let all := completeBindings
              (pvars (openPattern leftNames t) ++
               pvars (openPattern rightNames u)) bs
            [par (applyDot all (openWith leftNames k))
                 (applyDot all (openWith rightNames l))]
        | none => []
      else []
  | _, _ => []

def cstep (p : Pattern) : List Pattern :=
  stepClosed (pvars (close p)) (close p)
```

Theorem 5.1 (The corrected step)**.** *Each claim holds on the fragment named.*

1. Unifier, all patterns: *`unify` returns a principal, idempotent unifier when one exists and fails otherwise (`unify_sound`, `unify_most_general`, `unify_idempotent`, `unify_eq_none_iff`); one pass of its dotted bindings is $P\dot{\sigma}$ (`applyDot_unify`).*
2. Alpha-invariance, all processes: *if $P \equiv_\alpha P'$ then* $\mathtt{cstep}\, P = \mathtt{cstep}\, P'$ *(`cstep_alpha_invariant`, `cstep_alpha_rule_invariant`).*
3. Alpha, tame processes: *on processes of the calculus fragment whose guards have guard-free patterns, the manuscript's renaming rule, closed under equivalence and contexts, is exactly equality of closed forms (`alpha_iff_alphaEq`). Outside them a code inside a process is indistinguishable from a bound occurrence (`alphaEq_not_alpha_with_code`).*
4. Soundness, the calculus fragment: *every $Q \in \mathtt{cstep}\, P$ is a step $P \longrightarrow Q$ of the relational rules with the uniform dot (`cstep_sound_uniform`).*
5. Completeness up to alpha, the closing fragment: *for every step $P \longrightarrow Q$ whose* Comm *instances are closing there is $Q' \in \mathtt{cstep}\, P$ with $Q' \equiv_\alpha Q$ (`cstep_complete`, `cstep_adequate`).*
6. Completeness up to renaming, guard-free patterns: *for every step $P \longrightarrow Q$ of the uniform dot on guard-free patterns there are $Q' \in \mathtt{cstep}\, P$ and a renaming ρ that fixes every free name of P, is injective on the other names of Q', and makes $\rho\, Q' \equiv_\alpha Q$ (`cstep_complete_residual`). The renaming comes from the fact that two principal unifiers agree up to a renaming of their residual variables (`principal_renaming`).*
7. Parallel composition: *modulo the commutativity, associativity and unit laws, applied to the composition being executed and never inside patterns, the step that fires* Comm *at any pair of guards and* Refl *at any reflected component is sound for calculus processes (`cstepBag_sound`), and on the closing fragment it is complete for the rules without* Refl *(`cstepBag_complete`). When* Refl *quotes the canonical representative of its reduct, the listener depends only on the class of the reduct (`reflect_respects_bagEq`), and the step is complete on the closing fragment for all four rules (`cstepCanon_complete`).*
8. Firing: *for any principal unifier of inputs that are apart, with first-order continuations and a store whose variables are apart from the patterns,* Comm *is one valid firing of a multiset rewriting step with an equation store; every solution of the resulting store reads each output as the uniform dot below, which is the dotted continuation wherever the unifier binds the variables it reads (`comm_adequacy_corrected`).*

Why the uniform dot. With the manuscript's dot of an arbitrary principal binding list, the reduct depends on how a most general unifier is oriented: for $\{X \mapsto Y\}$ and $\{Y \mapsto X\}$ the continuation X reads $@Y$ and X (`manuscript_dot_depends_on_orientation`), and the reduct $X \mid @X$ of $\mathsf{for}(X \leftarrow @\mathbf{0})\, X \mid \mathsf{for}(Y \leftarrow @\mathbf{0})\, Y$ is related by no renaming to any executable reduct, since one of its continuations is not quoted (`manuscript_dot_not_up_to_renaming`). The uniform dot reads $@Z \mid @Z$ for one residual name Z, whatever the orientation (`uniform_dot_up_to_renaming`). It agrees with the manuscript's dot wherever every read variable is bound (`uniformDot_eq_applyDot`), and every solution of the firing's store reads it, for any principal unifier (`firing_reads_uniform_dot`). The renaming of completeness cannot be injective on all names: the relational rule may choose as a residual name a free name of the surrounding process, which the executable, fresh for the whole process, never does (`residual_meets_context`).

Reflection modulo the monoid laws. A reflected listener quotes a process, and a partner can compare quoted processes. If the listener quoted whichever representative of the reduct a derivation produced, one class of reducts would give listeners that a partner tells apart (`representative_reflection_two_lis`

`partner_tells_listeners_apart`), and the order of a parallel composition would become observable. In the reflective calculi a name is a quoted process taken up to structural congruence, so that $P \equiv Q$ implies $@P \equiv_N @Q$ [13]; order is not observable while multiplicity is. Following that, the formalization's Refl quotes the canonical representative of the reduct's class: its closed components, sorted by an injective structural code, with their multiplicities (`canonicalBag`, `bagEq_iff_canonicalBag_eq`). From either order of the control's two inputs, every listener is the same one (`canonical_reflection_one_listener`). The canonical order is exact for ground names. A pattern that quotes a bag with variables is still compared by unification as written, and a partner quoting another order misses the listener (`written_order_partner_misses`); comparing such patterns modulo the laws needs unification modulo associativity and commutativity, which yields complete sets of unifiers rather than one, and is not part of the rules.

Patterns that quote guards. The executable step does not fire on patterns that quote guards, even when the two patterns are alpha-equivalent (`guard_quoting_partners_refused`). Unifying such patterns up to their bound names is the problem of nominal unification [16]. On closed forms it becomes first-order: closing commutes with a substitution of closed values that captures nothing (`closeIn_psubst_closed`), so such a substitution unifies two patterns up to alpha exactly when it unifies their closed forms, on which a bound name is a code compared as a constant (`alphaEq_psubst_iff`). The first-order unifier of the closed forms is principal among capture-free alpha-unifiers (`alpha_unifier_absorbs`); when it binds a pattern variable to a code that a quoted guard binds, no capture-free alpha-unifier exists (`no_alpha_unifier_of_escaping`). Extending the executable Comm and its admission to such patterns is the remaining step.

The naive reading. `Reduction.lean` keeps a naive executable reading of the rules, which unifies the patterns as written with a triangular binding list read in one pass and substitutes into both continuations without renaming the inputs apart. Kernel-computed examples separate it from the rules:

- *Capture.* $\mathsf{for}(a \leftarrow @\mathbf{0})\, a \mid \mathsf{for}(\mathbf{0} \leftarrow @\mathbf{0})\, a$ reduces to $@\mathbf{0} \mid @\mathbf{0}$, identifying the second input's free a with the first input's bound a; its alpha-variant reduces to $@\mathbf{0} \mid a$, which is the corrected reduct of both.
- *A triangular list read once.* $\mathsf{for}(f(X, X) \leftarrow @\mathbf{0})\, X \mid \mathsf{for}(f(Y, a) \leftarrow @\mathbf{0})\, \mathbf{0}$ reduces to $@Y \mid \mathbf{0}$, while every unifier reads $@a$.
- *No renaming.* $\mathsf{for}(X \leftarrow @\mathbf{0})\, X \mid \mathsf{for}(f(X) \leftarrow @\mathbf{0})\, \mathbf{0}$ has no naive reduct; its corrected reduct is $@f(z) \mid \mathbf{0}$ for a fresh z.
- *The occurs check defeated.* $f(X, g(Z), Z, X)$ and $f(f(Y), Y, W, W)$ have no unifier, yet the naive unifier accepts them and the naive step fires.
- *Nested reflection.* The naive Comm-only lookahead has no result for a reflection of a reflection, so it misses a step of the relational rules (`naive_step_misses_nested_reflection`).

The defect is the missing convention, not only the unifier: the rules read literally, with no admission on Comm, relate the canary to the capturing reduct even with a principal unifier and alpha-equivalence (`literal_rule_admits_capture`), and the corrected rules exclude it under the convention alone (`capture_not_conventional_step`).

5.4 The MeTTa-Calculus as a GSLT

`GSLTInteraction.lean` (265 lines) gives the behavioral theory `mettaCalcGSLT`: its terms are the processes of the calculus fragment, its equations are alpha-equivalence, and its rewrites are the relational rules under the convention with the uniform dot. Equiv makes the rewrites respect

the equations on both sides. Every reduct of the corrected step that is a calculus process is a rewrite (`executable_step_to_mettaCalcGSLT`); on the closing fragment every rewrite is a reduct up to alpha (`closing_step_executes`), and on guard-free patterns every rewrite is a reduct up to renaming its residual names (`uniform_step_executes_up_to_renaming`). A second theory, `mettaCalcBagGSLT`, takes the monoid laws as its equations and the rules with canonical reflection as its rewrites, so that the order of a parallel composition is not observable; on the closing fragment every rewrite is a reduct of the step with canonical reflection, up to the laws (`closing_bag_step_executes`). The naive reading's capturing reduct of the canary is not a rewrite (`mettaCalcGSLT_no_capture`). A reduct may keep a guard of a continuation in closed form, and is then not itself a term of the theory. The authored language definition supplies the interactive presentation: the process sort, the parallel constructor and the symmetric COMM rewrite, a base interaction whose one premise is a relation query. The theory the generic engine presents from the authored definition has closed ground terms as its carrier, which contain no pattern variable, so it can express neither binding nor capture and is not used as the behavioral theory. The direct contact of two guarded parties through the located-transaction layer takes its premise as a parameter; with the corrected premise, inputs apart and the principal unifier of their patterns completed to name every pattern variable, every contact is a root COMM of the relational rules with the uniform dot (`correctedContact_step`).

5.5 Premise Intermediate Representation and Adequacy

`Premises.lean` (95 lines) declares a `PremiseProgram`—a datalog-style intermediate representation (IR) with two relations and two builtins:

$$\mathit{mettaComm}(t, u, P, Q, P', Q') \qquad \mathit{mettaStepNoReflect}(\mathit{src}, \mathit{tgt})$$

$$\mathit{mettaCommWitness}(t, u, P, Q) \rightsquigarrow \mathit{MRef}(P', Q') \qquad \mathit{mettaCommOnlyStep}(\mathit{src}) \rightsquigarrow \mathit{tgt}$$

with machine-checked stratification and wellformedness proofs.

`Adequacy.lean` (757 lines) answers the contract with the corrected COMM. A rule sees only its own bag, so the names a residual must avoid cannot come from the rule; the runtime supplies the free names of the process being stepped, which is how the corrected step chooses its fresh names too.

Theorem 5.2 (Premise–Runtime Adequacy)**.** *The following are proven:*

1. *`runtime_covers_declared_relations`, `runtime_covers_declared_builtins`: every declared relation and builtin is implemented by the runtime dispatcher.*
2. *`comm_witness_adapter_spec`: the witness for $\mathit{mettaCommWitness}(t, u, P, Q)$ is the corrected COMM of the two inputs, closed, opened apart with names fresh for its four arguments and the supplied names, unified, and dotted once.*
3. *`comm_witness_sound`: at a root redex of calculus terms, on any channel, every witness is a step of the relational rules under the convention.*
4. *`stepViaPremiseContract_sound`: every reduct of the run of the authored rules on a calculus process is a step of the relational rules modulo the monoid laws. The proof goes through the engine's relational specification of its fuel-bounded run and inverts the authored rules' matches (`engine_sound`).*
5. *`stepViaPremiseContract_complete_comm`: if two elements of one bag communicate on the closing fragment, the run produces the reduct, the other elements kept, up to the monoid laws.*
6. *`premise_contract_comm_canary`, `premise_contract_no_capture`, `premise_contract_comm_bagEq`: the authored rules reduce the canary to its corrected reduct in both orders of the bag and never*

to the capturing one, and each of their reducts is the corrected step's reduct up to the monoid laws.

Outside the fragment of the completeness item the run stops short: two inputs in different bags have no reduct, although the monoid laws bring them together (`nested_bag_missed`), and a reflection whose process is itself a reflection has none, because the contract's Refl premise is the declared Comm-only lookahead (`nested_reflection_missed`). Without the supplied names a witness is fresh for its redex only, and its residual name can be a free name of the surrounding bag (`supply_keeps_residuals_fresh`).

5.6 Interoperability with ρ-Calculus

`Interoperability.lean` (319 lines) provides a shared-fragment translation from the quote/drop/parallel subset of MeTTa-calculus into ρ-calculus. The translation `toRhoSharedProc?` succeeds on the translatable fragment and fails gracefully outside it.

For any successfully translated process, the file proves path-bisimulation self-witnesses, intrinsic rho-side inversion, and shared-core forward/backward simulation:

- `shared_translation_pathBisim_self`: the translated image is path-bisimilar to itself under the ρ-calculus `BisimulationKit`.
- `shared_translation_esBisim_self`: the corresponding (E, S)-bisimilarity corollary.
- `rhoSharedCore_step_inversion`: intrinsic inversion of `RhoSharedCoreReduces`.
- `rhoSharedCore_stepStar_inversion`: intrinsic inversion of `RhoSharedCoreReducesStar`.
- `sharedCore_step_forward_restricted`: one-step forward simulation on the quote/drop/parallel shared-core fragment.
- `sharedCore_step_backward_restricted`: source-anchored one-step backward simulation.
- `sharedCore_step_bisimulation`: combined one-step forward+backward shared-core bisimulation correspondence.
- `sharedCore_stepStar_forward_restricted`: reflexive-transitive forward simulation.
- `sharedCore_stepStar_backward_restricted`: reflexive-transitive backward simulation.
- `sharedCore_stepStar_bisimulation`: combined star-level forward+backward shared-core bisimulation correspondence.

Thus, forward and backward simulation are now formalized for translated shared-core quote/drop/parallel steps (and their star closure). Remaining future work is to extend this from shared-core to the full MeTTa fragment (including Comm/Refl) and prove an assumption-free cross-language bisimulation theorem.

5.7 Regression and Paper Map

`PaperMap.lean` (190 lines) provides a theorem-level index mapping each clause of the specification [11] to its Lean proof. Its reduction canaries are kernel computations of the corrected step: the canary reduces to exactly $@\mathbf{0}|a$ (`paper_comm_positive`), its reflection emits the listener that quotes this reduct (`paper_refl_positive`), and mismatched patterns block both (`paper_comm_negative`, `paper_refl_negative`); `paper_comm_relational` derives the same reduct by the relational rules, and `paper_comm_sound`, `paper_comm_complete`, `paper_comm_alpha_invariant` and `paper_comm_unifier_princ` index Theorem 5.1. The index also wraps step and star forward simulation, backward simulation, intrinsic backward inversion, and combined bisimulation for the shared fragment (`paper_shared_to_rho_*` theorems). `CoreMain.lean` now also exposes an explicit HM endpoint map (`coreMain_hm_endpoint_recommendati` distinguishing global canonical wrappers (assumption-scoped predecessor-finiteness) from the assumption-free scoped SC-quotiented family. `Regression.lean` (79 lines) fixes the exact outputs of the corrected step on the positive and negative Comm and Refl canaries and in a parallel context: as a list, as a set, and modulo the monoid laws, where Comm fires at either order of the pair. `CorrectedControls.lean` holds the separating examples against the naive reading. `RelationNames.lean` (28 lines) is a single source of truth for relation and builtin string identifiers.

6 Modal Mu-Calculus and the Rho-Mu Bridge

The modal μ-calculus [8, 2] extends Hennessy-Milner logic with least and greatest fixed-point operators. Our formalization covers standard semantics (`ModalMuCalculus.lean`, 360 lines).

6.1 Syntax

Formulas use de Bruijn indices for bound variables:

$$\phi ::= \top \mid \bot \mid \neg\phi \mid \phi \wedge \psi \mid \phi \vee \psi \mid \langle a \rangle \phi \mid [a]\phi \mid \mu X.\phi \mid \nu X.\phi \mid X$$

where $\langle a \rangle$ is diamond (possibility), $[a]$ is box (necessity), μX is least fixed point, and νX is greatest fixed point.

6.2 Rho-Calculus as LTS

The ρ-calculus reduction relation $p \rightsquigarrow q$ is a labelled transition system (LTS) with states = patterns, actions = () (unlabeled), and transitions = $\rightsquigarrow$. This directly induces modal operators:

Definition 6.1 (Reduction-Based Modalities)**.**

$$\texttt{possiblyProp}\,\phi\,p := \exists q.\, p \rightsquigarrow q \wedge \phi\, q \quad \text{(diamond)}$$
$$\texttt{relyProp}\,\phi\,p := \forall q.\, q \rightsquigarrow p \to \phi\, q \quad \text{(box)}$$

Theorem 6.2 (Galois Connection IS Modal Duality)**.**

$$(\forall p.\, \textit{possiblyProp}\,\phi\,p \to \psi\,p) \iff (\forall p.\, \phi\,p \to \textit{relyProp}\,\psi\,p)$$

This is exactly $\langle\rangle\phi \subseteq \psi \iff \phi \subseteq [\,]\psi$.

The proof is by direct unfolding of definitions (`galois_connection` in `Reduction.lean`). This validates the OSLF construction [14, 22]: operational semantics naturally gives rise to modal logic.

6.3 Spice Calculus as Iterated Diamond

The Spice lookahead corresponds precisely to iterated modal operators:

$$\texttt{presentMoment}(p) \equiv \langle\rangle\phi \quad \text{(1-step)}$$
$$\texttt{futureStates}(p, n) \equiv \langle\rangle^n\phi \quad (n\text{-step})$$
$$\texttt{reachableViaStarClosure}(p) \equiv \nu X.\, \phi \vee \langle\rangle X \quad \text{(eventually)}$$

All three equivalences hold by definitional equality (`simp`). Precognitive agents computing Spice lookahead are computing μ-calculus temporal properties.

6.4 Conjecture: Rho Strictly More Expressive Than Mu

Conjecture 6.3 (ρ-Calculus Strictly More Expressive)**.** *There exists a property expressible in ρ-calculus that is not expressible in modal μ-calculus. Specifically: μ-calculus cannot express "this process has the capability to quote itself."*

The intuition is that μ-calculus formulas are determined entirely by LTS behavior (which transitions occur), while ρ-calculus processes can inspect their own syntactic structure via quote/unquote. Two processes $x[x]$ (self-quoting) and $y[z]$ (not self-quoting) are behaviorally indistinguishable—both are dead—yet differ structurally. No μ-formula should be able to distinguish them.

A proper formalization would require: (1) a coinductive bisimulation relation (not just deadlock equivalence), (2) a simulation theorem showing ρ can faithfully encode μ-calculus evaluation, and (3) the separation argument over the actual ρ-calculus types from `ProcessCalculi/RhoCalculus/`. This remains future work.

Remark 6.4 (Gödel Analogy). This is analogous to Gödel's incompleteness theorem: first-order logic cannot express "this formula is provable"; adding a provability predicate (reflection) gives strictly more expressive power. Similarly, μ-calculus cannot express "this process quotes itself"; ρ-calculus can, via the quote operator @.

7 Quantale-Valued Semantics

Standard μ-calculus has Boolean semantics. We generalize to *quantale-valued* semantics [8] where satisfaction takes values in a commutative quantale Q (`ModalQuantaleSemantics.lean`, 520 lines).

Definition 7.1 (QLTS)**.** A *quantale-labeled transition system* is a tuple $(S, A, \text{trans} : S \times A \times S \to Q)$ where Q is a commutative quantale with residuation.

Definition 7.2 (Quantale-Valued Satisfaction)**.**

$$\texttt{qSatisfies}(s, \phi \wedge \psi) = \texttt{qSatisfies}(s, \phi) \sqcap \texttt{qSatisfies}(s, \psi)$$
$$\texttt{qSatisfies}(s, \phi \vee \psi) = \texttt{qSatisfies}(s, \phi) \sqcup \texttt{qSatisfies}(s, \psi)$$
$$\texttt{qSatisfies}(s, \langle a\rangle\phi) = \bigsqcup_{s'} \text{trans}(s, a, s') \otimes \texttt{qSatisfies}(s', \phi)$$
$$\texttt{qSatisfies}(s, [a]\phi) = \bigsqcap_{s'} \text{trans}(s, a, s') \Rightarrow \texttt{qSatisfies}(s', \phi)$$
$$\texttt{qSatisfies}(s, \mu X.\phi) = \bigsqcap\{P \mid \texttt{transformer}(P) \sqsubseteq P\}$$
$$\texttt{qSatisfies}(s, \nu X.\phi) = \bigsqcup\{P \mid P \sqsubseteq \texttt{transformer}(P)\}$$

where $a \Rightarrow b := \bigsqcup\{z \mid z \otimes a \sqsubseteq b\}$ is left residuation.

Theorem 7.3 (Environment Monotonicity with Polarity)**.** *For formula ϕ and variable i with polarity $p \in \{\text{true}, \text{false}\}$: if $\rho_1(j) = \rho_2(j)$ for all $j \neq i$ and $\rho_1(i) \sqsubseteq \rho_2(i)$, then*

$$p = \text{true} \implies \mathit{qSatisfies}(\rho_1, \phi) \sqsubseteq \mathit{qSatisfies}(\rho_2, \phi)$$

$$p = \text{false} \implies \mathit{qSatisfies}(\rho_2, \phi) \sqsubseteq \mathit{qSatisfies}(\rho_1, \phi)$$

The proof (130 lines) proceeds by structural induction on ϕ, handling each constructor: base cases ($\top, \bot, X$) are immediate; negation flips polarity via antitonicity of residuation; conjunction/disjunction preserve polarity via $\sqcap/\sqcup$ monotonicity; diamond/box use multiplication and residuation monotonicity; and the fixed-point cases (μ/ν) shift the de Bruijn variable index, extend the environment, and show every pre/post-fixed point for ρ_2 is also one for ρ_1.

This enables the Knaster-Tarski theorem, proving that fixed points exist and are computable as limits of approximation sequences (`muApprox_mono`, `nuApprox_antimono`).

Example 7.4 (Instantiations)**.**

- **Probabilistic**: $Q = [0, 1]$ with standard multiplication.
- **PLN evidence**: $Q = \mathbb{R}^{\infty}_{\geq 0} \times \mathbb{R}^{\infty}_{\geq 0}$ (positive/negative evidence counts).
- **Fuzzy**: $Q = [0, 1]^n$ (multi-valued truth).
- **Tropical**: $Q = \mathbb{R}^{\infty}_{\geq 0}$ with min/+ operations.

8 Common Infrastructure

The three process calculi share approximately 40% of their reduction and congruence infrastructure. A shared library in `ProcessCalculi/Common/` (4 files, 246 lines) eliminates this duplication through typeclasses and generic lemmas.

8.1 Typeclasses

`Common/ProcessAlgebra.lean` (31 lines) defines:

Listing 10: Shared process algebra typeclasses

```
class HasPar (P : Type) where
  par : P -> P -> P

class HasNu (P : Type) where
  nu : P -> P

class HasNil (P : Type) where
  nil : P
```

`Common/Congruence.lean` (64 lines) adds `HasSC` for structural congruence and the generic pattern `reduces_modulo_sc`.

8.2 Reflexive-Transitive Closure

`Common/Star.lean` (148 lines) provides two flavors of reflexive-transitive closure:

- `RTClosure`: Type-valued (carries witness data).
- `RTClosureProp`: Prop-valued (for logical reasoning).

Generic lifters—`liftUnary`, `liftBinLeft`, `liftBinRight`—allow multi-step reduction to be lifted through any unary or binary context constructor.

8.3 Instance Registration

Each calculus registers instances:

- **π-calculus**: `HasPar`, `HasNil`, `HasSC` (wrapping the Type-valued SC in `Nonempty`).
- **ρ-calculus**: `HasSC` (Prop-valued SC directly).
- **MQ-calculus**: `HasPar`, `HasNil`, `HasNu`, `HasSC` (all four typeclasses).

All three share: structural congruence as an equivalence relation with parallel commutativity/associativity and scope extrusion (unified via `HasSC`); and the COMM rule shape (sender + receiver $\to$ result), differing only in payload consumption (substitution in π/ρ, measurement in MQ).

9 Summary and Future Work

9.1 Summary

We have presented a comprehensive Lean 4.28 formalization of five process-algebraic systems and three cross-calculus bridges, totaling 17,821 lines with zero `sorry` and zero custom axioms (Table 1).

Key results include:

1. The Galois connection in ρ-calculus *is* modal diamond-box duality (Theorem 6.2), validating the OSLF construction.
2. Weak operational correspondence for restriction-free π-processes (`calculus_weak_correspondence_full_encod` forward simulation (Theorem 3.1), backward admin reflection with three-branch decomposition (reflexive, admin-canonical, or reflected π-step), and their composition into a single theorem—under restriction-freeness and Barendregt hygiene hypotheses, totaling 2,745 lines.
3. Constructive both-outcomes witness for MQ measurement branching, with backend-parametric denotational semantics and post-collapse normalization.
4. We conjecture that ρ-calculus is strictly more expressive than μ-calculus (Conjecture 6.3): reflection cannot be captured by behavioral observation.
5. Complete quantale-valued μ-calculus semantics with environment monotonicity (Theorem 7.3), enabling Knaster-Tarski fixed-point computation across probabilistic, fuzzy, tropical, and PLN-evidence domains.
6. Spice calculus n-step lookahead is exactly iterated diamond $\langle\rangle^n$, proving that precognitive agents compute μ-calculus temporal properties.
7. Classical Hennessy-Milner equivalence (observational indistinguishability iff bisimilarity) proved assumption-free for finite-carrier SC-quotiented canonical ρ-relations, and under predecessor-finiteness for the global relation. Forward image-finiteness is auto-discharged for all `LanguageDef`-based relations. A structural sufficient condition for predecessor-finiteness (`LangPredFiniteSafe` [22]) identifies exactly which languages admit the global HM iff without quotienting; the ρ-calculus is provably excluded.
8. MeTTa-calculus with executable COMM/REFL reduction, premise-IR adequacy bridge, and a shared-fragment translation into ρ-calculus with forward/backward shared-core simulation and bisimulation at both step and step* levels (Section 5).

9.2 Future Work

1. **Unrestricted π-to-ρ bisimulation**: extend the weak correspondence from restriction-free processes to full π-calculus (including ν and replication), which requires encoding name-server apparatus into the theorem statement. Remove Barendregt side conditions via explicit alpha-renaming. Package a single assumption-minimal weak bisimulation iff.

2. **Global Hennessy-Milner for ρ-calculus**: lift the finite-carrier SC-quotiented HM iff to the global unquotiented ρ-relation. A decidable sufficient condition for predecessor-finiteness (`LangPredFiniteSafe` [22]) now provides the positive direction, but the ρ-calculus provably fails it (COMM erases the channel name). The remaining path is quotienting to a finite-image semantics or proving an alternative finiteness argument that accounts for name erasure.

3. **Full MeTTa$\leftrightarrow\rho$ bisimulation**: extend the shared-core (quote/drop/parallel) forward/backward simulation to Comm and Refl fragments; remove the source-anchored restriction by proving translation injectivity.

4. **Multi-qubit MQ gates**: extend gate semantics beyond single-wire action; prove collapse preservation under composition and measurement idempotence.

5. **PLN temporal bridge**: prove that PLN temporal operators correspond to modal formulas over a time-indexed LTS.

6. **OSLF integration for MQ**: instantiate MQ processes as an OSLF language instance with $\Diamond \dashv \Box$ Galois connection.

7. **Spice-MQ bridge**: connect ρ-calculus Spice n-step lookahead to MQ measurement prediction.

References

[1] H. P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, revised edition, 1984.

[2] Julian Bradfield and Colin Stirling. Modal μ-calculi. In Patrick Blackburn, Johan van Benthem, and Frank Wolter, editors, *Handbook of Modal Logic*, pages 721–756. Elsevier, 2007.

[3] Arthur Charguéraud. The locally nameless representation. *Journal of Automated Reasoning*, 49(3):363–408, 2012.

[4] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. *Indagationes Mathematicae*, 34:381–392, 1972.

[5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[6] Simon J. Gay and Rajagopal Nagarajan. Communicating quantum processes. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 145–157, 2005.

[7] André Joyal, Mogens Nielsen, and Glynn Winskel. Bisimulation from open maps. *Information and Computation*, 127(2):164–185, 1996.

[8] Dexter Kozen. Results on the propositional μ-calculus. *Theoretical Computer Science*, 27:333–354, 1983.

[9] Rasmus Kjær Lybech. *Encodability and Separation for a Reflective Higher-Order Calculus*. PhD thesis, IT University of Copenhagen, 2022. PhD Thesis.

[10] Alberto Martelli and Ugo Montanari. An efficient unification algorithm. *ACM Transactions on Programming Languages and Systems*, 4(2):258–282, 1982.

[11] L. Gregory Meredith. A symmetric reflective higher order concurrent calculus with backchaining. Unpublished specification, 2024.

[12] L. Gregory Meredith. How the agents got their present moment. Preprint, 2026.

[13] L. Gregory Meredith and David F. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.

[14] L. Gregory Meredith and Mike Stay. Operational semantics in logical form. 2020. Draft / technical report. See also Williams & Stay, "Native Type Theory," arXiv:2102.04672.

[15] Robin Milner. *Communicating and Mobile Systems: the π-Calculus*. Cambridge University Press, Cambridge, 1999.

[16] Andrew M. Pitts. Nominal logic, a first order theory of names and binding. *Information and Computation*, 186(2):165–193, 2003.

[17] J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965.

[18] Davide Sangiorgi and David Walker. *The π-Calculus: A Theory of Mobile Processes*. Cambridge University Press, 2001.

[19] Mike Stay and L. Gregory Meredith. The MQ-calculus: A calculus for quantum processes involving intermittent measurement. Working draft, 2026.

[20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020.

[21] Christian Urban, Stefan Berghofer, and Michael Norrish. Barendregt's variable convention in rule inductions. In *Automated Deduction – CADE-21*, volume 4603 of *Lecture Notes in Computer Science*, pages 35–50. Springer, 2007.

[22] Zar and Oruži (Claude Anthropic). Verified operational semantics in logical form: A Lean 4 formalization of the OSLF algorithm. Mettapedia Technical Report, 2025. leanOSLF.pdf.